Spring 26 CSCI 240 – PA 5

General Trees & Binary Trees

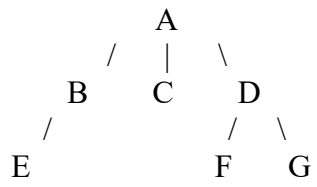
Feel free to discuss and help each other out but does not imply that you can give away your code or your answers! Make sure to read all instructions before attempting this lab.

New: You can work with a lab partner and each one must submit the same PDF file (include both names in the submission file). Each person must include a brief statement about your contribution to this assignment.

You must use an appropriate provided template from Canvas and output "Author: Your Name(s)" for all your programs. If you are modifying an existing program, use "Modified by: Your Name(s)".

Exercise 1: Provide clear pseudocode for preorder traversal for general trees **without recursion** (*hint: use a stack*). For a list of children, you can access nodes from left to right or from right to left. **Trace your algorithm** for the following tree to confirm that it is working correctly:

Preorder: A B E C D F G

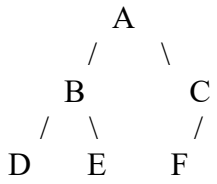


Exercise 2: Provide clear pseudocode for breadth-first (level order) traversal for general trees (*hint: use a data structure that you studied before*). **Trace your algorithm** for the tree in exercise 1 to confirm that it is working correctly:

Level order: A B C D E F G

Exercise 3: Use **LinkedBinaryTree** class from C++ or Java textbook (must use code from the textbook and make sure to understand the code) to create a simple test driver to perform some basic operations on a binary tree of strings. Once it is working correctly, modify LinkedBinaryTree class to support pre-order traversal. It is best to modify/override positions() to perform pre-order traversal instead of in-order traversal as given.

Test Case 1: Add code to construct a binary tree below and then perform a pre-order traversal on that binary tree. Confirm it is working correctly.



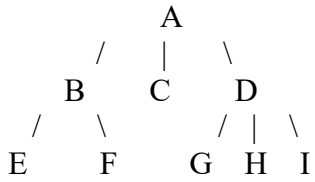
Pre-order traversal for binary tree above: A B D E C F

Test Case 2: Remove node C from the binary tree above and then perform a pre-order traversal on that binary tree. Confirm it is working correctly.

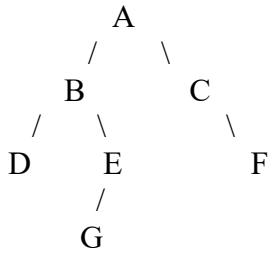
Pre-order traversal for updated binary tree without node C: A B D E F

Note: ***LinkedBinaryTree*** class *is implemented as an improper binary tree and all nodes hold data.*

Question 1: Provide preorder traversal, post-order traversal, and level-order traversal for the following general tree.



Question 2: Provide pre-order traversal, in-order traversal, post-order traversal, and level-order traversal for the following binary tree.



You can earn EC for only one of the options below:

Extra Credit Option 1: Provide pseudocode for post-order traversal for general trees without recursion. For a list of children, you can access nodes from left to right or from

right to left. **Trace your algorithm** for the tree in exercise 1 to confirm that it is working correctly.

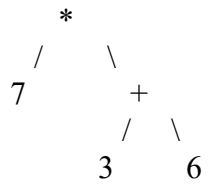
Post-order: E B C F G D A

Extra Credit Option 2: Add operation printExpession() to LinkedBinaryTree so it would print the binary tree without node B (deleted) in exercise 3 as show below.

((D B E) A F)

Construct the expression tree below and use printExpression() to print it. It would print

(7 * (3 + 6))



Fill out and turn in the PA submission file for this assignment (save as PDF format).